TAMPER-EVIDENT AUDIT LOG SERVICE

API and Design Study Guide

A practical explanation of the APIs, request flows, cryptographic choices, and trade-offs in this prototype.

Spring Boot | JWT | H2 / PostgreSQL | SHA-256 chain | Structured redaction | Signed export

> **What this guide answers:** what to call in Postman, what each response proves, how the hash chain detects tampering, why redaction does not invalidate a record, and what remains a prototype limitation.

Recommended reading order

1. Start with the end-to-end flow and Postman sequence.
2. Understand the hash chain before reviewing retention and redaction.
3. Finish with export verification and production trade-offs.

Local base URL: `http://localhost:8080`. The service assigns event timestamps in UTC. Current automated verification: 33 tests.

1. System at a glance

Layer	Responsibility	Why it exists
Controllers	HTTP routes, validation, request/response mapping.	Keeps the API surface separate from cryptographic business logic.
HashingAuditLogService	Creates hashes, verifies records, performs redaction/archive/export rules.	One place owns the integrity invariants.
JDBC repository	Stores ordered entries and JSON metadata.	Uses persistent H2 locally and PostgreSQL in production.
JWT service/filter	Issues and validates bearer tokens.	Prevents anonymous access to audit operations.

> **Timestamp decision:** the server assigns the UTC timestamp when it accepts an event. This prevents callers from deliberately backdating audit records. If source-system time is needed, retain it separately in the payload.

2. Authentication and starting the service

Run locally

```
mvnw.cmd spring-boot:run
```

The default profile is `local`. It uses a persistent H2 file database at `./data/audit-log`. Local credentials are development-only: `admin` / `change-me-local`.

Get a token

Method	URL	Authentication
POST	`/api/v1/auth/token`	Public

```
{
  "username": "admin",
  "password": "change-me-local"
}
```

Copy `accessToken` from the response. For every protected endpoint, Postman should send:

```
Authorization: Bearer <accessToken>
```

> **Common 401 cause:** when Postman uses the *Bearer Token* Authorization type, paste only the token into the Token field. Do not paste `Bearer <token>`, or the header becomes `Bearer Bearer ...`.

3. Core API reference

Purpose	Method and URL	Notes
Append event	`POST /audit` also `/api/v1/audit-logs`	Creates a new, immutable chain entry. Returns 201.
Query active events	`GET /audit`	Filters are optional and combine with AND.
Verify full chain	`GET /audit/verify`	Reads all records, including archived ones.
Archive by policy	`POST /audit/retention/archive?olderThanDays=90`	Soft archives only; does not remove chain evidence.
Redact fields	`PATCH /audit/records/{id}/redactions`	Redacts selected scalar JSON Pointer paths.
Export bundle	`GET /audit/export?actorId=user-42` `GET /audit/export?resourceId=customer-9`	Specify exactly one filter.

Create an audit event

```
POST /audit

{
  "eventType": "RECORD_UPDATED",
  "actorId": "user-42",
  "resourceType": "customer",
  "resourceId": "customer-9",
  "payload": {
    "changedFields": ["email", "phone"],
    "source": "admin-ui",
    "accountNumber": "1234567890",
    "person": { "name": "Jane Doe", "ssn": "111-22-3333" }
  }
}
```

Required fields are `eventType`, `actorId`, `resourceType`, `resourceId`, and an object `payload`. The response includes `id`, `timestamp`, `previousHash`, and `hash`.

Query events

```
GET /audit?actorId=user-42&resourceType=customer&resourceId=customer-9
          &eventType=RECORD_UPDATED&from=2026-09-01T00:00:00Z
          &to=2026-09-03T23:59:59Z&page=0&size=25
```

All query filters are optional. `from` and `to` are inclusive UTC times. Pages are zero-based; default size is 50 and the maximum is 100.

4. The normal request flow

1. **Authenticate.** Call the token endpoint and store `accessToken` in Postman.
2. **Check the initial state.** Call `GET /audit/verify`. An empty chain is valid.
3. **Write.** Create one or more events with `POST /audit`.
4. **Read.** Query by actor/resource/event type and inspect pagination.
5. **Verify.** Call `GET /audit/verify`; expect `valid: true`.
6. **Exercise Scenario B.** Redact data, archive eligible records, export a bundle, and verify again.

Verify response meaning

```
{
  "valid": true,
  "checkedRecords": 2,
  "firstInconsistentRecordId": null,
  "violationType": null,
  "message": "All audit events are intact"
}
```

Violation type	What it means
`PREVIOUS_HASH_MISMATCH`	A record does not point to the expected predecessor hash.
`HASH_MISMATCH`	Hash-bound event metadata or the declared payload root does not match the stored event hash.
`PAYLOAD_COMMITMENT_MISMATCH`	The visible payload, redaction metadata, salts, or commitments no longer fit the original payload commitment.

5. Why the hash chain detects tampering

```
First entry:
previousHash = GENESIS
hash = SHA-256(canonical event fields + payloadCommitment + timestamp + GENESIS)

Second entry:
previousHash = hash of first entry
hash = SHA-256(canonical event fields + payloadCommitment + timestamp + previousHash)
```

Every new entry carries the prior entry's hash. Altering an old record changes what its hash should be. The next record still references the original hash, so verification finds the first mismatch. The service checks all records in their stored chain sequence.

> **Why hash a payload commitment instead of raw JSON?** It gives a stable, deterministic cryptographic representation of the structured payload and makes later field-level privacy operations possible. The commitment root is still part of the record hash, so payload tampering remains detectable.

Direct database tampering test

The automated suite creates an event through the API, uses JDBC to change its stored H2 `payload_json`, then calls `GET /audit/verify`. The endpoint returns `valid: false` and `PAYLOAD_COMMITMENT_MISMATCH`. This demonstrates the assignment's required datastore-tampering scenario.

6. Retention: archive without a false chain break

Call:

```
POST /audit/retention/archive?olderThanDays=90
```

The service marks records older than the calculated cutoff with `archivedAt`. It does not physically delete them.

Question	Decision	Why
Are archived records shown in normal queries?	No.	Operational views can hide old data after policy archival.
Are archived records checked by verification?	Yes.	They remain historical evidence; omission would create a false break or hide tampering.
Is archive state included in the original event hash?	No.	It is legitimate lifecycle metadata, not the original assertion about what happened.
Are archived records included in export?	Yes, when they match the requested filter.	Export completeness must not depend on ordinary-query visibility.

> **Prototype trade-off:** the retention window is supplied per request through `olderThanDays`. A production system would normally add an approved policy configuration, scheduled execution, legal holds, and an audit event for the archival action.

7. Structured redaction: privacy without losing evidence

The problem

If the original record hash includes `accountNumber: "1234567890"`, simply replacing it with `[REDACTED]` changes the data and invalidates the hash. Keeping it indefinitely may violate privacy requirements.

The chosen scheme

```
At insert time, for every scalar payload leaf:
  random 128-bit salt
  commitment = SHA-256(salt + ":" + canonical scalar value)
  payloadCommitment = SHA-256(sorted map of JSON Pointer path -> commitment)
  event hash includes payloadCommitment

At redaction time:
  payload value -> [REDACTED]
  keep original field commitment
  delete that field's salt
```

Redact in Postman

```
PATCH /audit/records/{recordId}/redactions
```

```
{
  "fields": ["/accountNumber", "/person/ssn"]
}
```

What verification can prove after redaction	What it deliberately cannot prove
The visible marker is present, the original commitment is still part of the payload root, the salt is gone, and the original event hash/chain remain intact.	The original secret value. Salt destruction makes later offline guessing impractical but also makes original-value re-verification impossible by design.

> **Privacy limitation:** JSON structure and field names remain visible, and application memory, database logs, backups, or snapshots may retain old bytes. Stronger erasure uses per-field encryption plus cryptographic destruction of the field's encryption key.

8. Verifiable export

Choose exactly one filter:

```
GET /audit/export?actorId=user-42
GET /audit/export?resourceId=customer-9
```

The bundle is self-contained. It contains the selected records, every chain link as a payload-free proof, the chain head, a canonical SHA-256 digest, and an Ed25519 signature/public-key metadata.

Why include the full chain proof?

A bundle containing only matching records could silently omit one matching event. The proof includes all chain entries, so a recipient can apply the declared actor/resource filter independently and check that the matching proof IDs exactly equal the exported record IDs.

Recipient verification order

1. Canonicalize the unsigned bundle and recompute its SHA-256 digest.
2. Confirm it equals `bundleDigest`.
3. Validate the signing-key fingerprint and verify the Ed25519 signature.
4. Recalculate each proof event hash from `GENESIS` to `chainHeadHash`.
5. Validate each exported record's commitment/redaction data and membership in the proof.
6. Apply the declared filter and compare IDs for completeness.

> **Authenticity limitation:** the signing key is generated per process in this prototype. The included public key detects bundle alteration, but it does not by itself prove the identity of the exporter. Production needs a durable KMS/HSM key and an independently trusted key fingerprint.

9. Configuration, monitoring, and production choices

Area	Local	Production expectation
Database	Persistent file-backed H2.	PostgreSQL with externally supplied credentials and managed migrations.
JWT secret and credentials	Development defaults.	`JWT_SECRET`, `AUTH_USERNAME`, and `AUTH_PASSWORD` from a secret manager.
Monitoring	Health and info endpoints.	Health, liveness, readiness, metrics, and Prometheus access protected by network policy/authentication.
API docs	`/swagger-ui.html` and `/api-docs`.	Keep public only if organizational policy allows it.

Operational URLs

```
GET /actuator/health
GET /actuator/health/liveness
GET /actuator/health/readiness
GET /actuator/info
```

```
GET /actuator/metrics        (production)
GET /actuator/prometheus     (production)
```

10. Key trade-offs to explain in a review

Choice	Benefit	Cost / future improvement
Server timestamps	Prevents API callers from backdating events.	Does not prove the real-world event time; retain source time separately when necessary.
SHA-256 predecessor chain	Simple, explainable tamper evidence.	Multi-instance writers need database locking, optimistic retries, or a sequencer.
Soft archive	Hides expired records from normal views without erasing evidence.	Not a complete legal purge or backup-expiry solution.
Salted commitments + salt destruction	Supports redaction without breaking evidence and discourages guessing.	Cannot later prove the original redacted value; field names/shape remain.
Signed export	Detects altered bundle data and omissions.	Process-local key must become KMS/HSM-managed for production identity assurance.
JWT authentication	Protects APIs with stateless bearer tokens.	No fine-grained roles, revocation, refresh flow, or identity-provider integration yet.

11. Scenario C: compliance-reporting design

Scenario C is a design exercise, not an additional required code feature. The proposed direction is to represent account access as `resourceType=account`, use a stable internal `resourceId`, and capture the access outcome, purpose, source system, and correlation ID in the payload.

> Before building regulator-specific reports, obtain answers on what counts as access, regulator/jurisdiction, report format, retention/legal hold, recipients, required evidence, and approval workflow. The prototype documents these questions and deliberately scopes out unsupported compliance behavior rather than inventing policy.

12. Practical Postman checklist

1. Request a JWT token and set `accessToken`.
2. Call `GET /audit/verify`.
3. Create an event with `POST /audit`; save its returned `id`.
4. Query it using actor/resource filters.
5. Redact `/accountNumber` and `/person/ssn`; verify again.
6. Export using exactly one actor/resource filter.
7. Call archive with a valid retention window; new records may produce an archived count of zero.
8. Call verification one final time. It should remain valid after legitimate redaction and archival.

> **Final result:** the system supports append-only evidence, searchable audit history, explicit privacy transformations, and independently checkable exports while documenting the operational controls that a production deployment still needs.

Prepared from the current Audit Log Service implementation and repository documentation. This guide is a study aid; production policy and compliance decisions require organization-specific approval.